# I-BSIT-B Code Lab: Guide for Students and Professors

Oct 31, 2026 · @Carl

## 1. What Code Lab is

I-BSIT-B Code Lab is a code editor that runs in a web browser. Students write and run Python and C++ with nothing to install, and after one download it works with no internet.

Live site (deployed on Vercel by Carl Ianne): [i-bsit-b-web-ide-for-python-and-cpp.vercel.app](). Section 9 says exactly what has and has not been checked on that live copy.

### What it does

- **Editor:** the full VS Code editor (Monaco), or a simple editor if Monaco cannot load. It is the default on touch screens.
- **Runs Python 3.12** for real, inside the browser, using Pyodide 0.26.4.
- **Runs a simplified C++** using JSCPP 2.0.9. It is not a full compiler (see section 10).
- **Saves files on the student's own device** in the browser's database (IndexedDB). Nothing is uploaded, and the screen says so.
- **Cheat sheets:** 30 Python and 31 C++ topics, sorted from easiest to hardest, each tagged Must-know, Good to know or Nice to have. A **Copy to editor** button inserts the example.
- **Presets:** 20 commented example programs per language (hello world, loops, ASCII art and more).
- **Themes:** dark gray with forest green (default), or white with Google-style block colors.
- **Installable and offline:** a visible Install button, and an update notice when a new version is deployed.

### What it is not

- It is not a full C++ compiler, so some valid C++ will not run here.
- It has no accounts, no server and no sharing. A file lives only in the browser where it was made.

### How to read this guide

- **First-year students:** read sections 1, 2, 4 and 12 first.
- **Professors:** read sections 5 to 8 and 11 first. They cover how it was built, what was tested, and what was not.

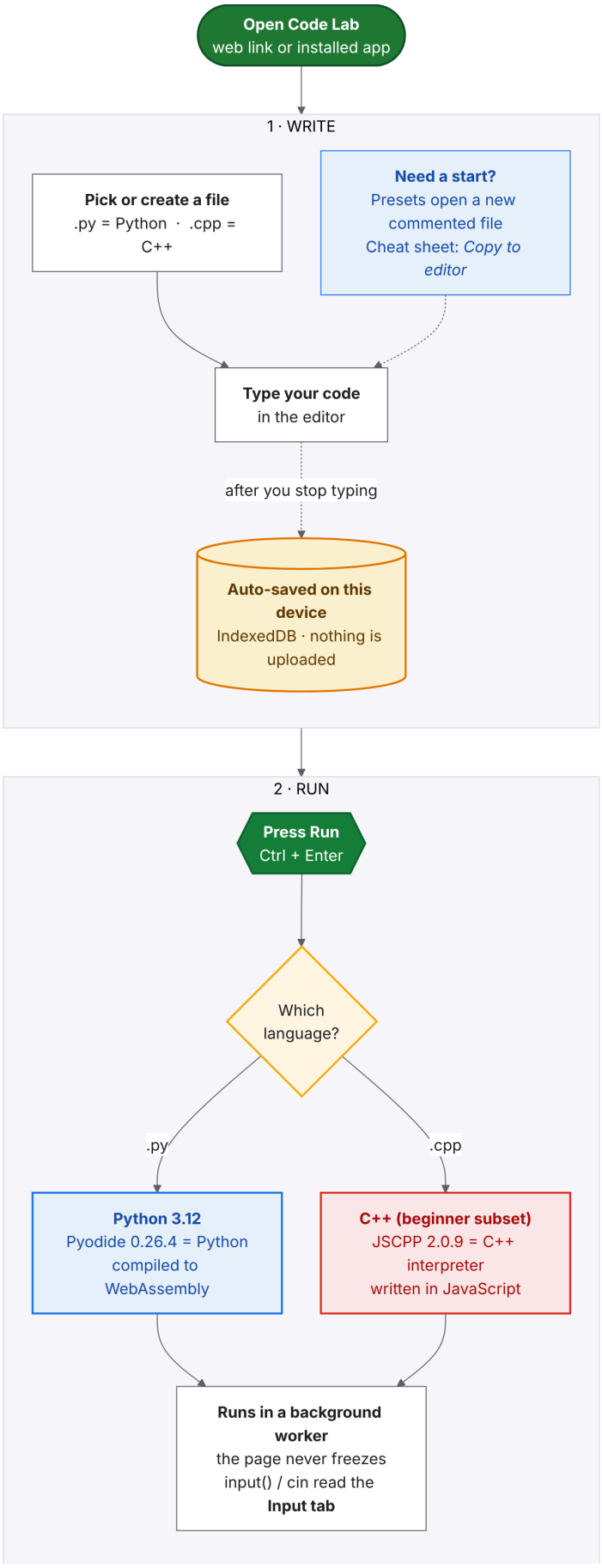
## 2. How it works

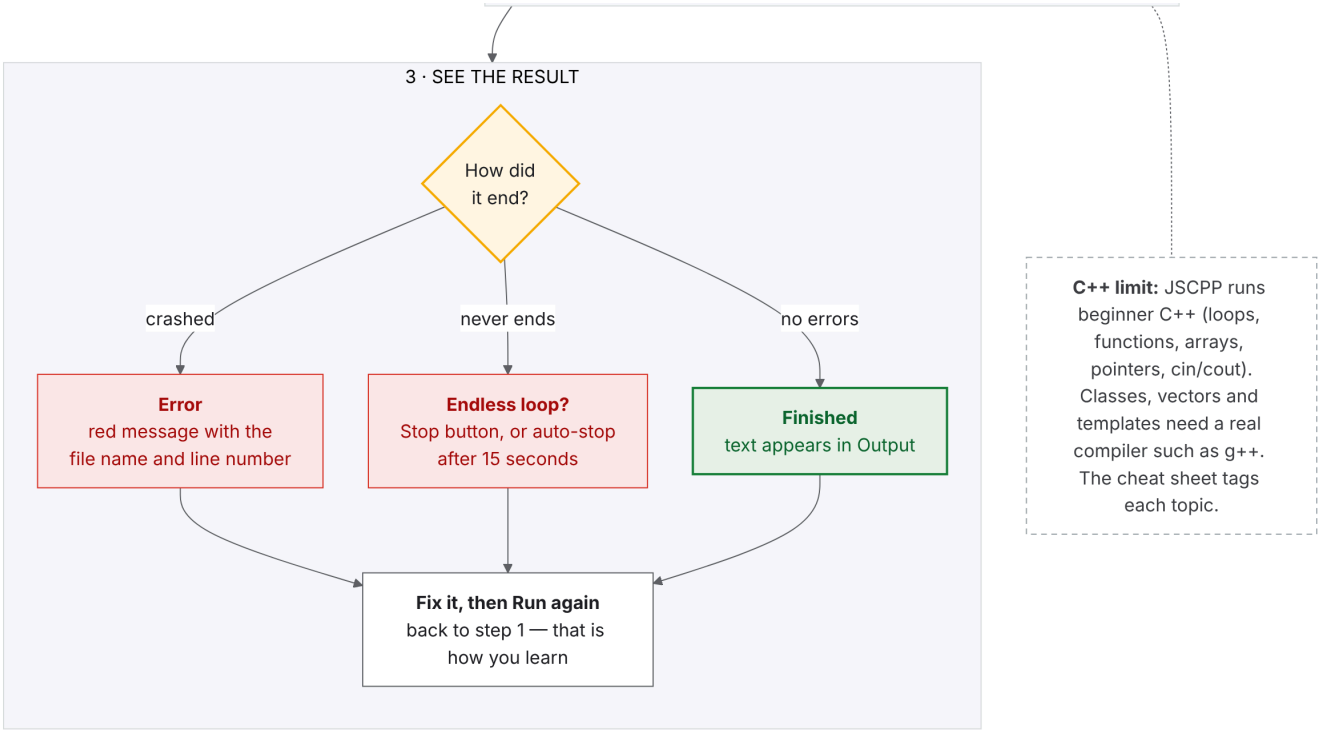
Everything happens inside the student's browser. There is no server that runs code or stores files; Vercel only hands out the website's files.

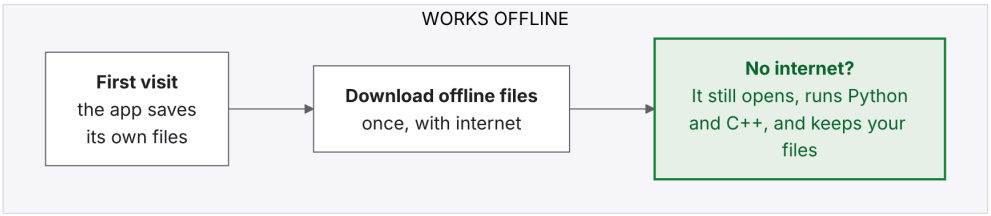
**From typing to output**

1. **Open the site.** The browser loads the page and scripts. A service worker quietly saves a copy for offline use.
2. **The app starts** (`main.js`). It opens the browser database, creates the editor, and shows the student's files.
3. **The student types.** The editor announces "something changed". The autosaver waits 0.6 seconds after typing stops, then saves that file.
4. **The student presses Run** (Ctrl+Enter). The app saves first, then sends the code and the Input tab text to a background worker for that language.
5. **The worker runs the program** and sends the printed text back a piece at a time. The page shows it in the Output tab.
6. **The run ends** in one of four ways: finished; an error shown in red with the file name and line; Stop pressed; or the 15-second limit reached. A program that prints more than 100,000 characters is also stopped.

A picture of this journey, made for students and professors, is in the project folder: `docs/how-code-lab-works.png` (its editable source is `docs/how-code-lab-works.mmd`).


Open Code Lab
web link or installed app
1 · WRITE
Pick or create a file
.py = Python · .cpp = C++
Need a start?
Presets open a new commented file
Cheat sheet: Copy to editor
Type your code
in the editor
after you stop typing
Auto-saved on this device
IndexedDB · nothing is uploaded
2 · RUN
Press Run
Ctrl + Enter
Which language?
.py
.cpp
Python 3.12
Pyodide 0.26.4 = Python compiled to WebAssembly
C++ (beginner subset)
JSCPP 2.0.9 = C++ interpreter written in JavaScript
Runs in a background worker
the page never freezes
input() / cin read the Input tab




**Why a background worker?** A program with an endless loop would freeze a normal web page. A worker is a separate thread, so the page stays alive and the Stop button can simply end the worker. This was tested with endless Python and C++ loops.

### How offline works

- On the first visit the service worker saves about 107 small files: the page, styles, scripts, cheat sheets and presets.
- The big engine files (Python is about 13 MB) are saved the first time they are used, or all at once with the **Download offline files** button.
- After that, the app opens and runs code with no internet. A test switches the network off, reloads, and runs Python.

### Where the data lives

Files are stored in IndexedDB, a database built into every modern browser. Clearing the browser's site data deletes them, which is why the app has a **Back up all files** button. In a private window the browser may refuse to save, and the app warns about it.

# 3. Map of the code

The code is split into small files grouped by job, and the groups are stacked in layers. A file may only use files in the same layer or below it, and a test fails if someone breaks that rule.

| Layer | Folder (in `public/js/`) | Its one job |
|---|---|---|
| 0 (bottom) | `config.js`, `core/` | Settings, engine versions, small helpers (page lookups, the message bus) |
| 1 | `storage/` | Saving files in IndexedDB, backup and restore |
| 1 | `content/` | Reading the cheat sheets and presets |
| 1 | `runners/` | Running Python and C++ in background workers, with Stop and time limits |
| 1 | `editor/` | Monaco and the simple editor, behind one shared set of methods |
| 1 | `pwa/` | Offline status, install button, update notice |
| 2 | `ui/` | Drawing things: file list, output panel, menus, dialogs, theme, toasts |
| 3 | `app/` | Deciding what happens: file controller, run controller, one setup file per feature |
| 4 (top) | `main.js` | Starts everything in order; 46 lines and no feature logic |

Outside `public/js/`:

- `public/data/` holds the cheat sheets (Markdown) and the presets.
- `public/sw.js` is the service worker that makes offline work.
- `scripts/build.mjs` copies the engines and writes the offline file list.
- `third_party/jscpp/` holds the C++ engine file and its license.
- `tests/` holds the automatic tests.

**Parts talk through a message bus.** The editor must be able to say "the student pressed Ctrl+Enter" before the run controller exists. So parts announce events (`run-requested`, `editor-changed`, `file-opened`) and `app/connectEvents.js` holds the one list of who reacts to each. To see what a feature does, read that file first.

**Rule for adding a feature:** write one new module and add one or two lines in `main.js`. Existing features stay untouched, which is how the project meets the "do not break other features" requirement.

## 4. The essential code blocks

Six short blocks carry most of the app's important behavior. Each one is shown as it exists in the project, followed by why it was built that way.

### 4.1 The message bus ( `core/bus.js` )

```
emit(name, ...args) {
  for (const listener of [...(listeners.get(name) || [])]) {
    try {
      const result = listener(...args);
      if (result && typeof result.catch === 'function') result.catch((error)
=> console.error(`[bus] "${name}" listener failed`, error));
    } catch (error) {
      console.error(`[bus] "${name}" listener failed`, error);
    }
  }
},
```

**Why:** one part shouts an event and any other part can listen, so neither needs to know the other exists. The `try/catch` means a crash in one listener never stops the others from hearing the message.

### 4.2 The autosaver ( `app/autosaver.js` )

```
async function saveIfDirty() {
  const id = pendingId;
  if (!id) return;                 // nothing waiting
  const snapshot = pendingSnapshot;
  pendingId = null;                // clear first, so an edit made DURING the
save is not lost
  pendingSnapshot = null;
  await save(id, snapshot);
}
```

**Why:** it remembers which file changed and the text at that moment. An earlier version read the screen at save time, which could save one file's text into another file (see section 8, bug B).

### 4.3 One write at a time (`storage/fileStore.js`)

```
let writeQueue = Promise.resolve();
function inOrder(task) {
  const result = writeQueue.then(task);
  writeQueue = result.catch(() => {}); // a failed write must not block the
ones after it
  return result;
}
```

**Why:** an update reads the file, changes one part and writes it back. Two overlapping updates (code and Input) both read the old file, and the second write erased the first. The queue makes writes wait their turn.

### 4.4 Running safely with Stop and a time limit (`runners/workerRunner.js`)

```
case 'started':
  ...
  active.runTimer = setTimeout(
    () => finish({ status: 'timeout' }, { killWorkerToo: true }),
    limits.runTimeoutMs,
  );
```

**Why:** the program runs in a worker. If it takes longer than the limit (15 seconds), or the student presses Stop, the page ends the worker. The next run starts a fresh worker, so an endless loop can never freeze the app.

### 4.5 The C++ engine loader (`runners/jscppLoader.js`)

```
if (!hadWindow) scope.window = scope;
if (!hadDocument) scope.document = {};
try { loadFile(); }
finally {
  if (!hadWindow) delete scope.window;
  if (!hadDocument) delete scope.document;
}
```

**Why:** JSCPP's browser file only defines itself when it sees a page (a `window` with a `document`). A worker has neither, so C++ would have failed to start. The loader pretends for a moment, then removes the pretend objects. This was found while testing, not by guessing (section 8).

### 4.6 The service worker (`sw.js`)

```
const saved = await caches.match(request, { ignoreSearch: true });
if (saved) return saved;                        // 1. a saved copy: instant,
works offline
try {
  const response = await fetch(request);        // 2. otherwise ask the
internet
  ...                                           //    and save engine files
for next time
  return response;
} catch {
  ...                                           // 3. offline and not saved:
show the app or a clear message
}
```

**Why:** every file the page asks for passes through here first. This is what makes the app open with no internet. Saving a copy has its own `try/catch`, so a full disk can no longer turn a good download into a false "you are offline" error.

## 5. How it was built, step by step

**Who did what.** Carl Ianne, a first-year BSIT student, wrote the requirements, made the decisions (for example, removing Java) and deployed the site. The code, tests and documents were written by Claude, an AI assistant made by Anthropic, working in a cloud workspace from Carl's prompts. Carl is new to Claude and to deploying apps, so the work was done in rounds: build, measure, then improve. Section 6 describes how Carl directed the AI.

1. **Requirements (Oct 28).** One long message set the whole scope: an offline editor for I-BSIT-B, Python and C++, saving on the device, cheat sheets, presets, themes, an install button, clean modular code and a Vercel deployment.
2. **Scope cut.** Java was removed on purpose to keep the app light, and Carl asked for beginner step-by-step deployment instructions (`docs/STEP-BY-STEP.md`), including GitHub.

3. **First build, in layers.** Settings and helpers first, then storage, the editor, the runners for Python and C++, the cheat sheets and presets, the offline service worker, and the screens on top. Each layer was written to depend only on the layers below it.
4. **Content written and checked.** The cheat sheets and presets are plain Markdown and files, so they can be edited without touching the code. Their examples were run for real (section 7).
5. **Interrupted and resumed.** The first working session hit its message limit just as it was finishing. Carl told the next session not to start over but to list the existing files and continue, and the project was finished from where it stood.
6. **Flowchart.** Carl asked for a picture to teach from; it became `docs/how-code-lab-works.png`.
7. **Objective rating.** Before changing anything, Carl asked for an honest maintainability rating. It came back as 7.5 out of 10, with `main.js` and the file controller named as the weak spots.
8. **Refactor to about 8.5.** `main.js` went from one 263-line function to a 46-line list of connections, and autosaving became its own small tested unit. Guard tests were added so the structure cannot quietly get messier.
9. **Discrepancy checks.** Code, page, documents and data were compared against each other. This found the two engine bugs and the mis-tagged C++ topics in section 8.
10. **User-experience fix.** Carl asked that students understand the C++ limits, so three warnings were added: a badge, a one-time notice, and a hint when a run fails.
11. **Code review.** A review for race conditions and security found two data-loss bugs in autosave. They were reproduced in a real browser, fixed, and covered by tests.
12. **Deployment and this guide.** Carl deployed the project on Vercel and asked for this document.

**What this means for reading the project.** The AI wrote most of the code, so the learning value is in understanding it, not in having typed it. Section 11 suggests how a professor can check that understanding.

## 6. How Carl directed the AI

Carl guided the AI the way a project lead guides a developer: he set the goal, asked for work in stages, and checked each result before approving the next step. This section describes that approach by stage, in his role as the person in charge, instead of repeating his messages.

| Stage | What Carl asked the AI to do | What it led to |
|---|---|---|
| 1. Set the goal | Build an offline-capable code editor for his class in Python and C++. It should save work on the student's device, include cheat sheets and ready-made examples, have light and dark themes and an install button, use clean modular code, never break other features, and be deployed on Vercel. | The first build of the whole app, in layers |
| 2. Narrow the scope | Drop Java to keep the app light, and teach him deployment step by step as a beginner, including GitHub. | Java removed; `docs/STEP-BY-STEP.md` written |
| 3. Protect the work | When the first session ended early, he told the AI to continue from the existing files instead of starting over, and to list them first. | The files were listed and the unfinished work (tests and checks) was completed |
| 4. Ask for teaching material | A concise flowchart that first-year students and professors can learn from. | `docs/how-code-lab-works.png` |
| 5. Ask for an honest measurement first | An objective maintainability rating before any change was made. | A measured rating of 7.5 out of 10, with the weak spots named |

| Stage | What Carl asked the AI to do | What it led to |
|---|---|---|
| 6. Approve a bigger improvement round | He estimated the code could reach about 8.5 out of 10 and told the AI to go ahead. He also asked for the flowchart in the zip and README, cross-checks of the code, and clear Python and C++ versions. His goal was a prototype as product-ready as possible before manual testing. | The refactor, new tests, engine-version statements, and the two engine bugs found |
| 7. Question before deciding | He asked whether the bugs found were a concern, how accurate the app is for practice, and whether it warns about the C++ limits, each time asking for answers only, with no changes. | Honest answers: fixed bugs and real limits for the first, Python strong and C++ basic for the second, and warnings that existed but were easy to miss for the third |
| 8. Put users first | He wanted classmates to clearly understand the C++ limits. | The C++ badge, the one-time notice and the failed-run hint |
| 9. Ask for a deeper review | He asked the AI to analyze the code for race conditions, endless loops, hidden type problems, modular structure, comments and security, again without changing anything yet. | A review with real-browser reproductions; two data-loss bugs found |
| 10. Approve the fixes | He told the AI to go ahead with the recommended fixes. | Both bugs and a service-worker problem fixed, each with a new test |
| 11. Ask for this guide | A guide for students and professors covering how it works, how it was built, what was tested and what was not, and the live Vercel link. | This document |

**Pattern worth noticing.** Carl often asked for analysis separately from changes: questions first, building only after he approved. This kept him in charge of what changed, and it is a good habit for any project, with or without AI.

## 7. Testing and verification

As of the last full run before deployment, **111 automatic unit tests and 25 real-browser steps all passed**. All of this was run by the AI in its own cloud workspace, not on Carl's computer or on Vercel. The two tables below separate what was tested from what was not.

### What was tested

| Area | How it was tested | Result |
|---|---|---|
| Python examples | All 30 cheat-sheet examples and all 20 presets run in real Pyodide | Pass |
| Python version | A test asks the real engine for its version and compares it to what the app tells students | Python 3.12, matches |
| C++ examples | Every example and preset compiled with g++ 13 ( `-std=c++17 -Wall -Wextra` ) with no warnings | Pass |
| C++ in the browser engine | Every topic tagged "runs here" gives the same output in JSCPP as in g++; every topic tagged "needs g++" really fails in JSCPP | Pass, after one example and four tags were fixed |
| Saving and files | File store tests: create, rename, delete, duplicates, bad names, oversized files, backup and restore | Pass |
| Autosave races | Tests for editing code and Input together, and for a keystroke during a file switch, in the unit tests and in a real browser with real IndexedDB | Pass after the fixes |

| Area | How it was tested | Result |
|---|---|---|
| Running code | Real browser: Python and C++ run in workers, Python error shows file and line, input works, endless loops stop with Stop and by the time limit | Pass |
| Offline | Real browser: the service worker saves the app, the network is switched off, the page reloads and runs Python | Pass |
| Service worker | The real `sw.js` run in a fake browser, including a full-storage case | Pass |
| Screens | Cheat sheet, presets, new/rename/delete, theme remembered after reload, install help, C++ badge, notice and hint | Pass |
| Phone layout | A narrow browser window: panels become overlays and nothing scrolls sideways | Pass (emulated) |
| Code structure | No circular imports, layers only point downward, size limits on functions, `main.js` stays short | Pass |
| Consistency | Every element id in the code exists in the page; every button is used; every import resolves; numbers in the README and dialogs match the data | Pass |
| Linter (ESLint) | Strict rules | 0 errors, 14 warnings (all "function is long or complex") |
| Clean build | The project zip was unpacked in an empty folder and the real build ran | Pass |

### What was NOT tested

| Area | Why it matters |
|---|---|
| The full Monaco editor | It could not be downloaded in the AI's workspace, so only the simple editor was ever run. If Monaco fails, the app falls back to the simple editor with a notice. It must be checked once on the live site. |
| The live Vercel site's behavior | A page fetch showed it is online and serves the right page, but could not run its scripts. Whether Python, C++, Monaco and offline work on the live copy has not been checked by the AI. |
| Real phones and other browsers | Only Chromium was used. Safari, Firefox, and Android and iPhone installs are untested. |
| "Install app" on a real device | Only the help dialog was tested. |
| Speed | How fast C++ runs on a slow phone, and how long the first load takes on a slow connection, were not measured. |
| Storage full in a real browser | Only simulated in a unit test. |
| Accessibility | No screen-reader or keyboard-only audit. |
| Security hardening | No Content-Security-Policy was added, and there is no `package-lock.json` yet (see section 10). |
| Independent review | The maintainability rating (7.5, then about 8.5) is the AI's own measurement, not an outside audit. |

**How to read "tested".** Passing tests show the checked behaviors work. They do not prove the absence of other bugs. Two real bugs (section 8) were found by a review and a reproduction, not by the first set of tests.

## 8. Bugs found and fixed

Seven real problems were found before Carl's students could meet them. The last three could silently lose a student's work or mislead them.

| # | Problem | What a student would have seen | How it was found | Fix |
|---|---|---|---|---|
| 1 | JSCPP's npm package does not include its browser file | The Vercel build would have failed | Checking what the package really contains | The official file was added to `third_party/jscpp/` with its license, and the build copies it |
| 2 | JSCPP does not define itself inside a worker | "C++ engine could not start" on every run | Reading how the file loads, then testing it in a real worker | `jscppLoader.js` (section 4.5) |
| 3 | Four C++ topics were tagged "may run here" but never ran: `std::string`, pass by reference, `struct`, `new/delete` | Errors on topics the app said might work | Running every non-"yes" topic in JSCPP | Retagged "needs g++"; a test checks tags in both directions |
| 4 | A decimal printed differently: `12.56636` in JSCPP, `12.5664` in g++ | Output that did not match the textbook | Comparing JSCPP with g++ output | The example was changed so both agree |
| 5 | **Bug A:** code and Input saved at the same moment overwrote each other | The new code was lost, the new Input kept | Review, then reproduced in a real browser | Writes now run one at a time (section 4.3) |
| 6 | **Bug B:** a keystroke during a file switch saved the other file's text | `a.py` ended up containing `b.py`'s text | Review, then reproduced in a real browser | The autosaver saves a snapshot taken at edit time (section 4.2) |

| # | Problem | What a student would have seen | How it was found | Fix |
|---|---|---|---|---|
| 7 | A full browser cache could cause a false "you are offline" message while online | An engine file failed to load with a wrong message | Reading the service worker, then a test with a simulated full cache | Saving a copy has its own `try/catch` (section 4.6) |

**Two honest notes.**

- Bugs A and B were rare: they needed a save collision or a keystroke within a few milliseconds of a file click. But the damage was total, so they were fixed before any manual testing.
- While testing the C++ hint, one new browser test failed because the hint text itself contained the word "fine", which the test was waiting for. That was a mistake in the test, not the app, and it was corrected.

## 9. Deployment on GitHub and Vercel

The project is published from a GitHub repository to Vercel, which builds it and serves it at [i-bsit-b-web-ide-for-python-and-cpp.vercel.app](). The full beginner steps are in `docs/STEP-BY-STEP.md`.

**How the build works** (settings in `vercel.json`)

1. Vercel runs `npm install`, which downloads Pyodide and Monaco.
2. Vercel runs `npm run build`. The script copies the engines into `public/vendor/` (JSCPP comes from `third_party/jscpp/`), and writes the offline file list with version numbers.
3. Vercel serves the `public/` folder. A new deploy changes the version numbers, so students receive the update, and see a "new version is ready" notice.

The big engine files are not uploaded to GitHub. `.gitignore` leaves them out, because Vercel rebuilds them. The site must be served at the root of its address, as it is on the `vercel.app` link.

**What was checked on the live site**

- A page fetch on Nov 3, 2026 reached the site. It returned the page titled "I-BSIT-B Code Lab" with the Run, Presets, Cheat sheet, theme, Install and More controls, the Files panel and the Output panel.
- That check could not run the page's scripts. It could not confirm that the editor drew, that Python or C++ ran, or that offline mode worked.

**Checklist for the first manual test of the live site**

- [ ] Open the link and wait about 10 seconds. The editor appears; if you see "The full editor could not load", Monaco failed and the simple editor is in use.
- [ ] Press Run on the starter Python file. "Hello!" appears and the run says "Finished".
- [ ] Open a preset in C++ and run it. The C++ notice appears once, and the badge says "C++ · simplified".
- [ ] Write `#include <vector>` in a C++ file and run it. The red error is followed by the explanatory note.
- [ ] Edit a file, reload the page, and confirm the edit was saved.
- [ ] In the ⋮ menu, open the saving and offline dialog and press Download offline files. Switch off Wi-Fi, reload, and run Python.
- [ ] Press Install app (Chrome or Edge on a computer, or Add to Home Screen on a phone).
- [ ] Repeat the first four checks on a phone and on one other browser (Firefox or Safari).
- [ ] Run `npm install` on your own computer once and commit the new `package-lock.json`, so every Vercel build uses the same package versions.

# 10. Limits and security

## Engine versions

| Language | Engine | Version | Meaning |
|---|---|---|---|
| Python | Pyodide 0.26.4 | Python 3.12 | Real CPython. The standard library works; extra packages, `turtle` and `tkinter` do not. |
| C++ | JSCPP 2.0.9 | No C++ version number | A simplified interpreter that follows the basics of old-style C++ (about C++98). It is not a compiler. |

## C++ limits

- **Runs here:** variables, loops, functions, arrays, pointers, `cin` and `cout`, `<cmath>` and `<cstring>`.
- **Does not run here** (checked topic by topic): `std::string`, references (`&`), `struct`, classes, `new` and `delete`, vectors, templates, exceptions and files.
- **Small output differences** can exist, such as decimal formatting. Only programs compared with g++ are known to match.
- **No compiler warnings,** and mistakes like reading past the end of an array behave differently from real C++.

The app warns students in three places: the file badge says "C++ · simplified", a one-time notice appears on the first C++ file, and a failed C++ run ends with a note saying the error may be an app limit, not their mistake. For graded work or the features above, students should use a real compiler such as g++.

## Security and privacy

### Already good (checked in the code)

- The app never builds HTML from text, so a file named `<script>` is harmless.
- It makes no third-party requests: no analytics, fonts or ads. Engines are served from the site itself.
- Restoring a backup validates every file name and text.
- It collects no personal data, has no accounts and has no server database.

### Worth knowing

- **Student code can reach the saved files and the network.** In testing, a Python program could see the browser's database and `fetch`. A classmate's "cool script" could read or delete files, or send them elsewhere. Students should not run code from strangers, and should use the backup button.
- **Shared school computers:** files stay in that browser profile for the next person. Students should use a private window (the app warns that nothing is saved there) or delete their files.
- **No Content-Security-Policy** is set. One that allows connections only to the site itself would block sending data out. It was not tried, because Pyodide needs relaxed rules and it would need testing.

- **No `package-lock.json` yet,** so package versions beyond the two pinned ones are not locked.

**Not needed for this kind of app:** logins, encryption, or server security. There is no server and no personal data.

## 11. For professors

This project was written mostly by an AI from a first-year student's prompts, so the useful question is not "who typed it" but "does the student understand it, and was it checked honestly?" This section gives a way to answer both.

**Suggested reading order (about 30 minutes)**

1. `public/js/main.js` : 46 lines that show the whole start-up in order.
2. `public/js/app/connectEvents.js` : the one table of "when this happens, do that".
3. `public/js/app/autosaver.js` and `storage/fileStore.js` : the two blocks behind the data-loss fixes.
4. `public/js/runners/workerRunner.js` : Stop, time limit and output limit.
5. `public/sw.js` : how offline works.
6. `tests/` : start with `fileController.test.mjs` and `architecture.test.mjs` .

**What the project shows well**

- Layered, modular structure that is enforced by tests, not only by habit.
- Tests that reproduce real bugs first, then pass after the fix.
- Honest labeling of what each C++ topic can and cannot do, checked against a real compiler.

**Where it is weaker**

- A few long functions remain (the ESLint warnings), such as the worker runner and the file store.
- It is plain JavaScript with no type checking.
- The full editor and the live deployment have not been verified by the author or the AI (section 7).

**Questions to check the student's understanding**

1. Why does the autosaver store a snapshot of the text, and not read the screen when it saves? What went wrong before?

2. Why do file writes wait in a queue? Draw two overlapping updates and show how one erased the other.
3. Why does code run in a worker? What would happen to the page with an endless loop if it did not?
4. JSCPP is an interpreter and g++ is a compiler. What can a student learn from each, and what can go wrong with the interpreter?
5. How does the service worker decide between a saved copy and the network? What happens offline?
6. Which of the "not tested" items in section 7 would you test first, and how?
7. The AI wrote most of this code. What would you check yourself before trusting it?

**Maintainability rating.** The AI rated the first version 7.5 out of 10 and the refactored version about 8.5. Both are the AI's own measurements, from function sizes, structure rules and tests. They are not an independent audit.

## 12. Glossary for first-year students

| Term | Plain meaning |
|---|---|
| PWA (Progressive Web App) | A website that can be installed like an app and can work offline |
| IndexedDB | A database built into your browser where the app saves your files on your own device |
| Service worker | A helper script that sits between the page and the internet and keeps saved copies of files, so the app opens offline |
| Web Worker | A background thread. Your program runs there, so the page never freezes |
| WebAssembly (Wasm) | A fast format that lets languages like Python run inside a browser |
| Pyodide | Python compiled to WebAssembly. It runs real Python 3.12 in the browser |
| JSCPP | A C++ interpreter written in JavaScript. It handles beginner C++ only |

| Term | Plain meaning |
|---|---|
| Interpreter vs compiler | An interpreter runs your code line by line as it reads it. A compiler turns the whole program into a machine program first and can warn about more mistakes |
| Module | One small code file with one job, which other files can import |
| Layer | A group of modules. Higher layers may use lower layers, never the reverse |
| Message bus | A shared board where parts announce events and others listen, without knowing each other |
| Autosave (debounce) | Waiting until you stop typing for a moment, then saving once, so it does not save on every key |
| Snapshot | A copy of the text taken at one moment, so later changes cannot alter what gets saved |
| Race condition | A bug where the result depends on which of two things happens first, such as two saves overlapping |
| Precache | The list of files saved ahead of time so the app works offline |
| Build | The step where Vercel prepares the site's files before serving them |
| Unit test | A small program that checks one piece of code gives the right result |
| Smoke test (browser test) | A test that opens the real app in a real browser and clicks through it |
| Linter | A tool that scans code for mistakes and risky patterns without running it |
| Repository (repo) | The folder of project files stored on GitHub, with its history |

*End of guide. Written on Nov 3, 2026. If the site or the code changes after that date, parts of this guide may be out of date.*